

Boku-nano

制限された日本語からPythonの関数を生成する小型言語モデル
(SLM)

Boku-nano：制限された日本語からPythonの関数を生成する小型言語モデル（SLM）

24種類の整数リスト操作を扱う、小型言語モデルをフルスクラッチ学習

使うとき

① 日本語の指示

整数リストxs から
偶数だけを残し、
各要素に k を加える
solve 関数を書いて
決められた操作・文型の範囲



② Boku-nano

15M パラメータ
Decoder-only Transformer

指示を読み、コードを生成



③ Python の関数

```
def solve(xs, k):  
    return [x + k for x in xs  
            if x % 2 == 0]
```

作り方と確かめ方

1

意味を定義

24操作を最大3つ
順序も含めて組み合わせる

2

教材を作る

日本語指示と正解コード
訓練データ 192,900件

3

モデルを学習

トークナイザもモデルも
既存の重みを使わず学習

4

実行して評価

生成コードの動作を確認
84,180 / 84,550問 合格

課題と狙い

抽出や順序の変更など、対象を24種類の操作に限定する

範囲を絞る

あらかじめ定めた整数リストの操作

操作を組み合わせて処理を表す

目指すモデル

学生一人でもフルスクラッチで作れる

小規模言語モデル（SLM）

別の用途への応用

関数の組み合わせで表せる処理なら、
同じ考え方を **NPCの挙動**などにも応用できる可能性がある

意味ASTの具体例

日本語：偶数だけを残して、各要素にkを加える

```
{  
  "sequence": [  
    {"filter": ["even"]},  
    {"map": ["add_k"]}  
  ]  
}
```

偶数の抽出 → kを加える、という処理の順序を保存

意味ASTから訓練データを自動生成・検証する

共通の処理の定義：意味AST

```
{"sequence": [  
  {"filter": ["even"]}, {"map": ["add_k"]} ]}
```

日本語の指示文

生成の手順

Qwenで各操作の日本語表現を生成。
ASTの順に結合し、テンプレートの
{operations} をルールベースで埋める。

生成例

整数リストxsと整数kを受け取り、
偶数だけを残し、各要素にkを加える
solve関数を書いてください。

検証済みのPythonコード

生成の手順

ASTの操作をPythonコードへ変換。
構文を確認し、実行結果を
参照インタプリタの期待値と照合する。

生成例

```
def solve(xs, k):  
    return [x + k for x in xs  
            if x % 2 == 0]
```

扱う範囲は24種類の操作

種類	操作数	例
条件に合う数を残す	10	偶数、奇数、正の値、kより大きい値
一つずつ値を変える	8	kを加える、2倍、絶対値、二乗
順序を変える	3	昇順、降順、現在の順序の反転
一部を取り出す	3	先頭k個、末尾k個、1個おき

最大3操作を組み合わせる、限定されたコード生成課題。

同じ意味から、日本語とコードを作る

段階	作るもの	例
① 意味を決める	操作と順序の記録	偶数を残す。その後、kを加える
② 指示を作る	日本語の説明	偶数だけを残して、各要素にkを加える
③ 正解を作る	Pythonコード	指示どおりに処理するsolve関数
④ 動作を確認する	入力と出力の比較	参照実装と結果が一致するか

指示と正解を、一つの意味に対応付ける。

操作の組み合わせは12,720種類

同じ操作を繰り返さず、順序を区別して数える。

操作の数	数え方	意味の種類
1操作	24	24
2操作	24×23	552
3操作	$24 \times 23 \times 22$	12,144
合計	$24 + 552 + 12,144$	12,720

同じ操作の繰り返しは、別のテストで確かめる。

一つの意味に、複数の日本語を用意する

共通の意味	指示の言い回し
偶数を残す	偶数だけを残す
偶数を残す	偶数の要素を取り出す
偶数を残す	偶数の値を抽出する

ルールと補助モデルを使い、意味を確認して教材に採用する。

訓練・検証・テストに意味を分けてから、表現を増やす。

コードの書き方にも変化を付ける

どちらも「偶数だけを残す」正しい書き方。

```
result = [x for x in xs if x % 2 == 0]
```

```
result = []  
for x in xs:  
    if x % 2 == 0:  
        result.append(x)
```

文字列が同じかではなく、動作が同じかを重視する。

学習に使った教材は192,900件

1件は「日本語の指示」と「正解コード」の組。

処理の長さ	訓練データの件数
1 操作	460
2 操作	8,680
3 操作	183,760
合計	192,900

約95%が3操作の教材。 単独操作の教材は少ない。

三つの大きさを、1 epochで比較する

パラメータは、学習で調整するモデル内部の数値。

呼び名	パラメータ数	今回の学習
1M	約100万	教材全体を1回使う
5M	約500万	教材全体を1回使う
15M	約1,500万	教材全体を1回使う

既存の学習済み重みを使わず、Transformerをゼロから学習する。

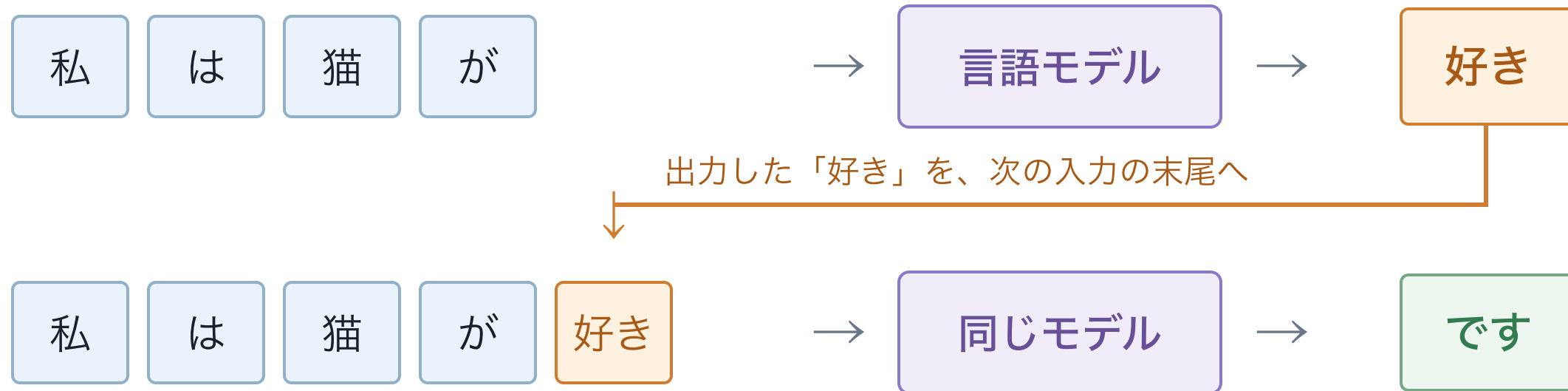
自己回帰：出力を次の入力に加えて繰り返す

GPTなどの言語モデルは、**直前までの言葉から次を予測**して文章を作る。

入力：ここまでの文章

予測するモデル

出力：次の言葉



学習では、文章の途中までを見せ、実際の次の言葉を当てるように学ぶ。

説明用に言葉で区切った例。実際はトークン単位で予測し、生成を繰り返す。

① 次のトークンは、候補ごとの確率分布で表す

```
# 今の入力から、次のトークンの点数を得る
# まずは temperature=1.0 として読む
logits = (
    model(input_ids).logits[:, -1, :]
    / temperature
)

# 候補を点数の高い順に並べる
sorted_logits, sorted_indices = torch.sort(
    logits, descending=True, dim=-1
)

# softmaxで、点数を確率分布に変換する
sorted_probabilities = torch.softmax(
    sorted_logits, dim=-1
)
```

Python比較評価から抜粋。コメントを追加し、改行を調整。
evaluate_boku_nano_comparison.py : 209-211行

最初から1つの答えが出るわけではない

モデルは、語彙の全トークンに対する **点数**（ロジット）を出す。

`[:, -1, :]` は、入力の**最後の位置**から予測する、全候補の点数。

softmaxで変換すると、各候補が0～1の値を持ち、**合計が1**になる。

候補	点数	確率
A	2	66.5%
B	1	24.5%
C	0	9.0%

計算例。この分布から**どの候補を選ぶか**が、次の処理。

② 選んだトークンを入力に加え、予測を繰り返す

```
# 日本語指示を含む入力ID列をコピーする
current = prompt_ids.copy()
generated: list[int] = []

# 勾配を計算せず、次のトークンを順に生成
with torch.inference_mode():
    for _ in range(min(
        max_new_tokens,
        model.config.context_length - len(current)
    )):
        values = torch.tensor(
            [current], dtype=torch.long)
        # この実装では最大点数の候補を選ぶ
        token_id = int(torch.argmax(
            model(values).logits[0, -1]).item())
        # 終了トークンなら生成を止める
        if token_id == eos_token_id:
            break
        # 選んだIDを出力と次回の入力へ追加
        generated.append(token_id)
        current.append(token_id)
```

Python推論から抜粋。コメントを追加し、改行を調整。
export_boku_nano_onnx.py : 232-241行

自己回帰の中心は `current.append`

1. **ここまでの入力**から次の候補を予測する。
2. 候補を1つ選び、**入力末尾に追加**する。
3. 伸びた入力を使って、**同じモデル**でもう一度予測する。

重みは更新せず、入力が1トークンずつ伸びる。

このコードはgreedy生成

`argmax` は最大の点数の候補を選ぶ。

softmaxを通して順位は同じなので、**greedyでは確率への変換を省ける**。

EOSまたは生成長の上限で止まる。

③ 温度は、確率分布の偏りを調整する

```
# 全候補のロジットを温度Tで割る
logits = (
    model(input_ids).logits[:, -1, :]
    / temperature
)

# 候補のIDを保持して点数順に並べる
sorted_logits, sorted_indices = torch.sort(
    logits, descending=True, dim=-1
)

# 調整後の点数をsoftmaxで確率にする
sorted_probabilities = torch.softmax(
    sorted_logits, dim=-1
)
```

evaluate_boku_nano_comparison.py : 209-211行。
Pythonのsampling処理では temperature > 0 が条件。

計算は **`softmax(logits / T)`**

各候補の点数を**Tで割ってから**、softmaxを適用する。

点数 [2, 1, 0] の例：

温度T	A	B	C
0.5	86.7%	11.7%	1.6%
1.0	66.5%	24.5%	9.0%
2.0	50.6%	30.7%	18.6%

小さいT：最大候補に確率が集中する。

大きいT：他の候補も選ばれやすくなる。

順位は変わらない。温度を上げても、正解率が上がるとは限らない。

T=0は除算せず、前ページのgreedy方式を使う。

④ top-pで、抽選する候補を絞る

```
# ... (ロジットの計算・softmaxまでを省略)
# 高確率の候補から累積確率を計算する
cumulative = torch.cumsum(
    sorted_probabilities, dim=-1)
# top-pに達した後の候補を除外する
remove = (
    cumulative - sorted_probabilities >= top_p
)
# 範囲外の候補の確率を0にする
sorted_probabilities = (
    sorted_probabilities.masked_fill(remove, 0.0)
)
# 残した候補の合計が1になるよう正規化
sorted_probabilities /= sorted_probabilities.sum(
    dim=-1, keepdim=True)
```

evaluate_boku_nano_comparison.py : 212-215行。
説明用コメントを追加し、改行・字下げを調整。

このPython実装はtop-pも使う

確率の高い順に足し、累積がtop-p以上になる最小の候補群を残す。

例：確率 [0.6, 0.3, 0.1]、top-p=0.8

候補	累積確率	残すか
A	0.6	残す
B	0.9	残す
C	1.0	除外

AとBの確率を再び合計1にすると、Aは約66.7%、Bは約33.3%になる。

top-p=1なら、正の確率を持つ全候補を残す。

⑤ 確率に従って選び、次の入力へ追加する

```
# 確率に従って1つ抽選する
sampled_positions = torch.multinomial(
    sorted_probabilities,
    num_samples=1,
    generator=generator,
)
# 並べ替えた位置から元のトークンIDに戻す
next_ids = sorted_indices.gather(
    1, sampled_positions).squeeze(1)
# ... (EOS判定・系列ごとの追加ID作成を省略)
# 次の入力に加えるIDをテンソルにする
appended = torch.tensor(
    append_values, dtype=torch.long, device=device)
# 入力列の末尾へ追加する
input_ids = torch.cat(
    (input_ids, appended[:, None]), dim=1)
```

evaluate_boku_nano_comparison.py : 216-222、237-238行。
省略を明示。コメントを追加し、改行・字下げを調整。

抽選は `torch.multinomial`

`num_samples=1` で、各入力について候補を1つ選ぶ。

確率が約66.7%と約33.3%なら、その比率に応じて選ばれる。毎回最大候補になるとは限らない。

選んだトークンを次の入力へ

`gather` で、点数順に並べた位置を元のトークンIDへ戻す。

終了判定などを行った後、`torch.cat` で入力の末尾に追加する。

伸びた入力で次を予測する。この繰り返しが、自己回帰生成。

五つのテストで、生成コードを実行する

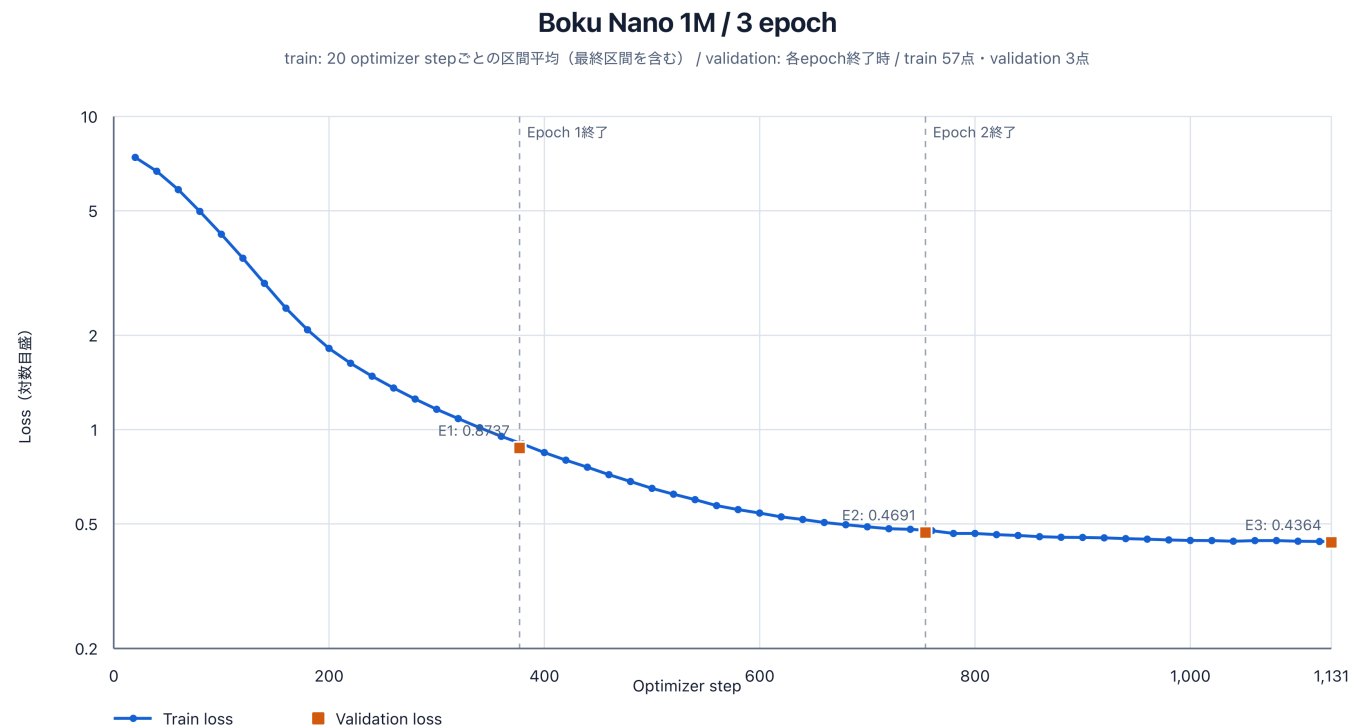
テスト	確かめること	指示の件数
通常	未学習の操作の並びを扱えるか	24,040
組合せ	訓練から隔離した操作対を組めるか	13,400
日本語の言い換え	未学習の30表現を理解できるか	30
同じ操作の反復	同じ操作を2回・3回実行できるか	23,040
境界値	空リストなどでも正しく動くか	24,040

1 指示につきコードを1候補生成し、全検査を通ったら合格。

通常と境界値では同じ指示を使い、検査する入力を変える。

1M : 3 epochまでの損失の推移

3 epochの学習。訓練損失は20 stepごと、検証損失は各epoch終了時の実測値。

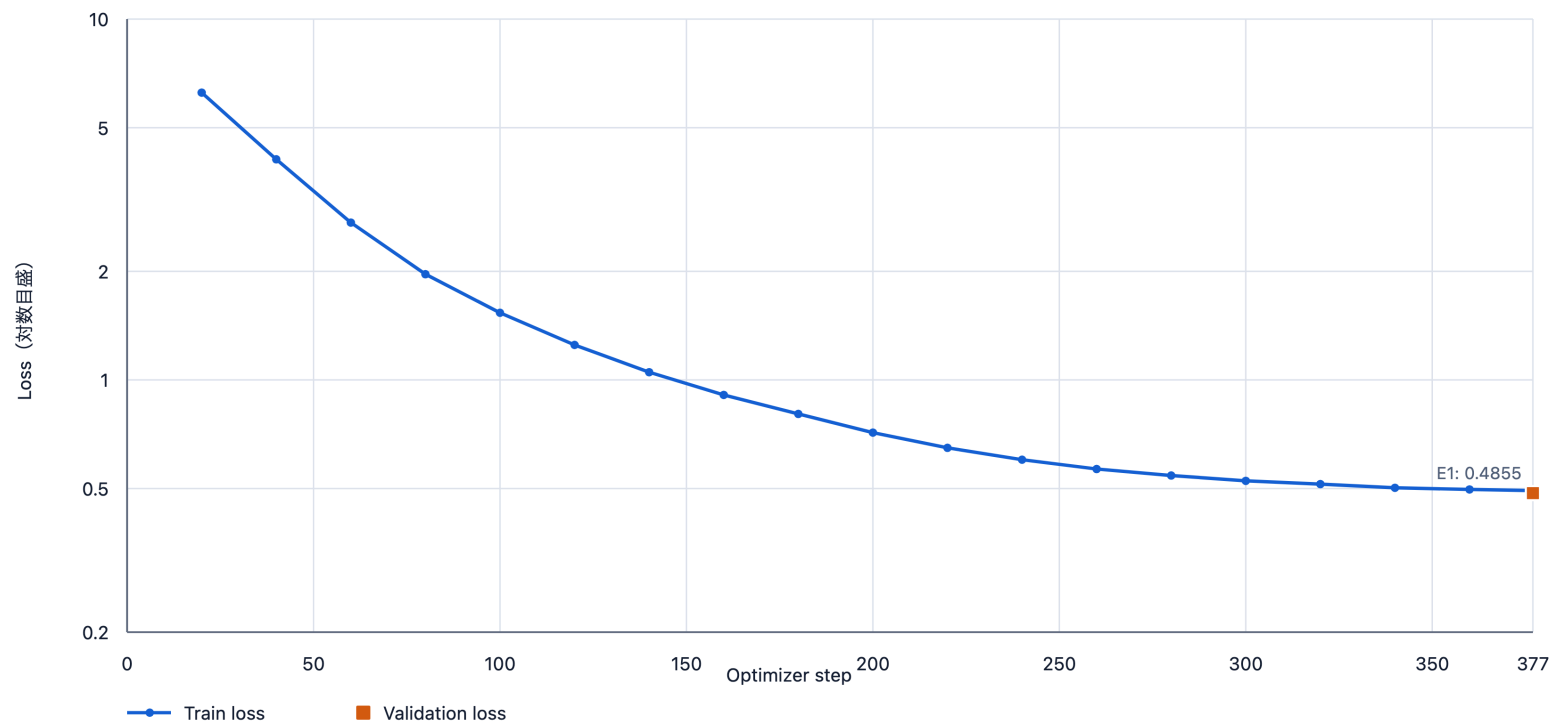


val loss : 1 epoch 0.8737 / 2 epoch 0.4691 / 3 epoch 0.4364

20 stepごとのvalidationは未記録。図は保存済みの3点を表示。

5M：損失の推移

1 epochの学習。訓練損失と、検証データで測った損失を示す。

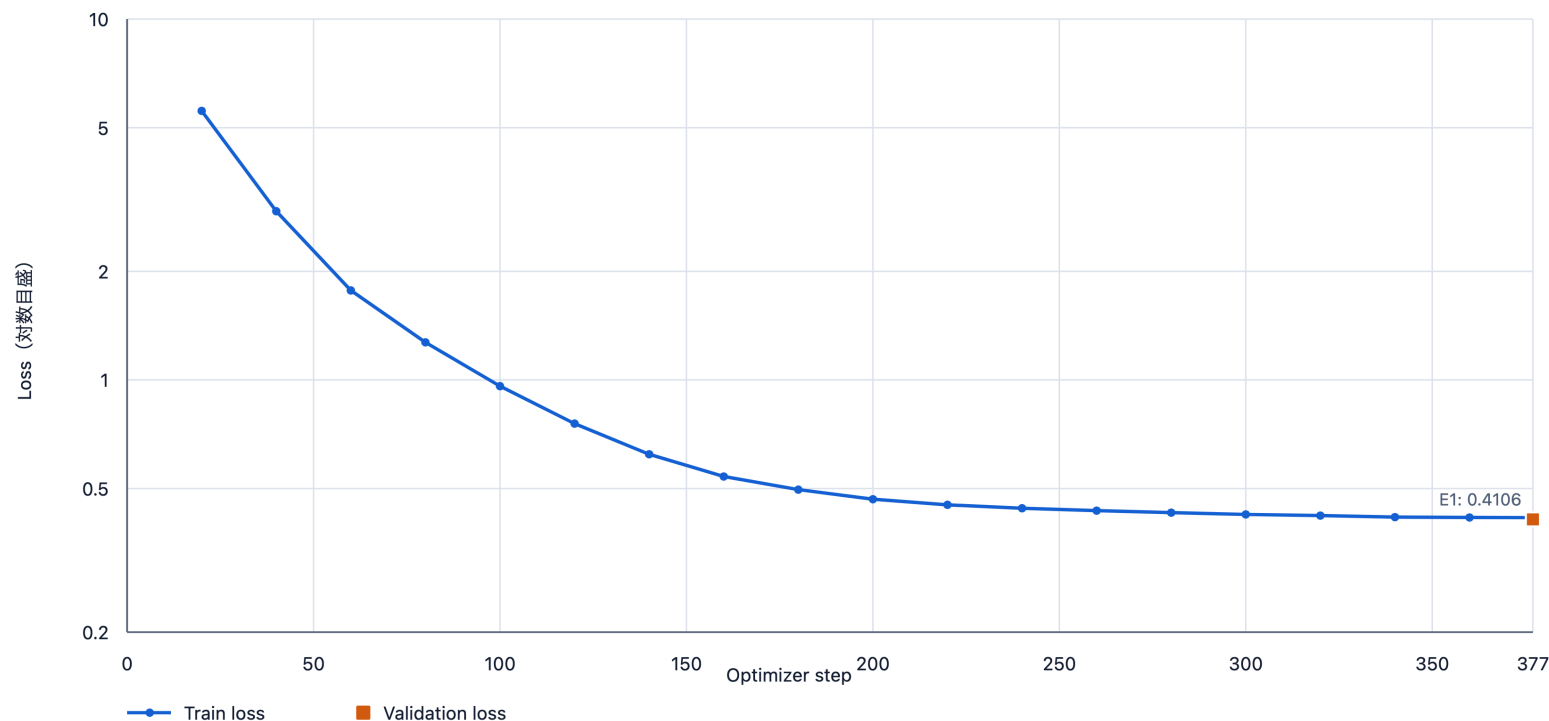


学習終了時の val loss : 0.4855

val lossは終了時の1点。途中の検証値は保存されていない。

15M：損失の推移

1 epochの学習。訓練損失と、検証データで測った損失を示す。



学習終了時の val loss : 0.4106

val lossは終了時の1点。途中の検証値は保存されていない。

1 epochの結果：モデル規模による違い

五つのテストを合計した参考値。分母は84,550件。

モデル	合格数	合格率
1M	1,467件	1.74%
5M	81,557件	96.46%
15M	84,180件	99.56%

今回の条件では、5Mと15Mは高い全体合格率を示した。

ただし、件数の少ない言い換えテストは全体値に表れにくい。

全体が高精度でも、言い換えには弱い

15Mモデル・1 epochのテスト別結果。

テスト	合格数	合格率
通常	24,003 / 24,040	99.85%
組合せ	13,188 / 13,400	98.42%
日本語の言い換え	15 / 30	50.00%
同じ操作の反復	22,971 / 23,040	99.70%
境界値	24,003 / 24,040	99.85%

全体の99.56%だけでは、日本語表現への弱さが見えない。

先行研究：日本語の書き方で正解率が変わる

Gan and Mori（2023）は、意味や指示に近い5種類のプロンプトを比較した。

プロンプトは、モデルに渡す指示文や質問文のこと。

日本語の課題	モデル	正解率の範囲
2文の意味関係を分類するJNLI	GPT-4	25.44～49.21%

意味が近くても、書き方によって性能が大きく変わる場合がある。

出典：[Gan and Mori, PACLIC 2023, Table 3](#)

先行研究：言い換えで回答の傾向も変わる

Takayama et al. (2026) は、日本語を含む「はい／いいえ」の質問を評価した。

表現の変え方	日本語での観察
同意を求める「～ですよ？」	道徳判断で「はい」が増える傾向
反対語への置き換え	答えの反転を考慮しても、一貫性が低下

同じ判断を求めても、表現によって答えが動く。

モデルの評価には、複数の言い回しを含める必要がある。

出典：Takayama et al., LREC 2026, Table 1

今回は、指示の冒頭だけを変えて比較する

条件	モデルに渡す指示
変更前	整数リストxsから偶数だけを残すsolve関数を書いてください。
変更後	整数リストxsに対して、偶数だけを残すsolve関数を書いてください。

後半の操作の指示をそろえ、冒頭だけが異なる**14操作**を比較。

同じモデルを使い、追加学習せず、同じ入力でコードを検査する。

5Mモデルでは、12件の合格が1件に減った

5M・1 epoch。比較した14操作の合格数。

指示の冒頭	合格数を表す図	合格数・合格率
整数リストxsから	■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ □ □	12 / 14・85.7%
整数リストxsに対して、	■ □ □ □ □ □ □ □ □ □ □ □ □ □	1 / 14・7.1%

■は合格1件、□は不合格1件。左から合格数を並べた図。

冒頭を変えただけで、ほとんど正しく生成できなくなった。

失敗例：偶数を残す指示を別の処理にした

変更後の指示：整数リストxsに対して、偶数だけを残すsolve関数を書いてください。

	指示どおりの処理	変更後に生成した処理
操作	偶数だけを残す	順序を反転し、k以上の値を残す
検査入力	xs = [0]、k = 1	同じ入力
出力	[0]	[]

構文は正しかったが、処理の意味が変わった。

表現への強さは、モデルによって違う

すべて1 epoch。冒頭だけを変えた14操作の合格数。

モデル	「xsから」	「xsに対して、」
5M	12 / 14	1 / 14
15M	12 / 14	12 / 14

15Mでは合格数を保った。すべてが同様に崩れるわけではない。

学習データの文型との結び付きが考えられる

訓練データで「整数リストxsに対して、」と始まる指示は84件。

その文型で学習した処理	件数
1 操作	0
2 操作	1
3 操作	83

この文型を、複数の操作と結び付けて覚えた可能性がある。

原因の確定には、教材を変えた追加比較が必要。

モデルが情報を読み取る仕組み

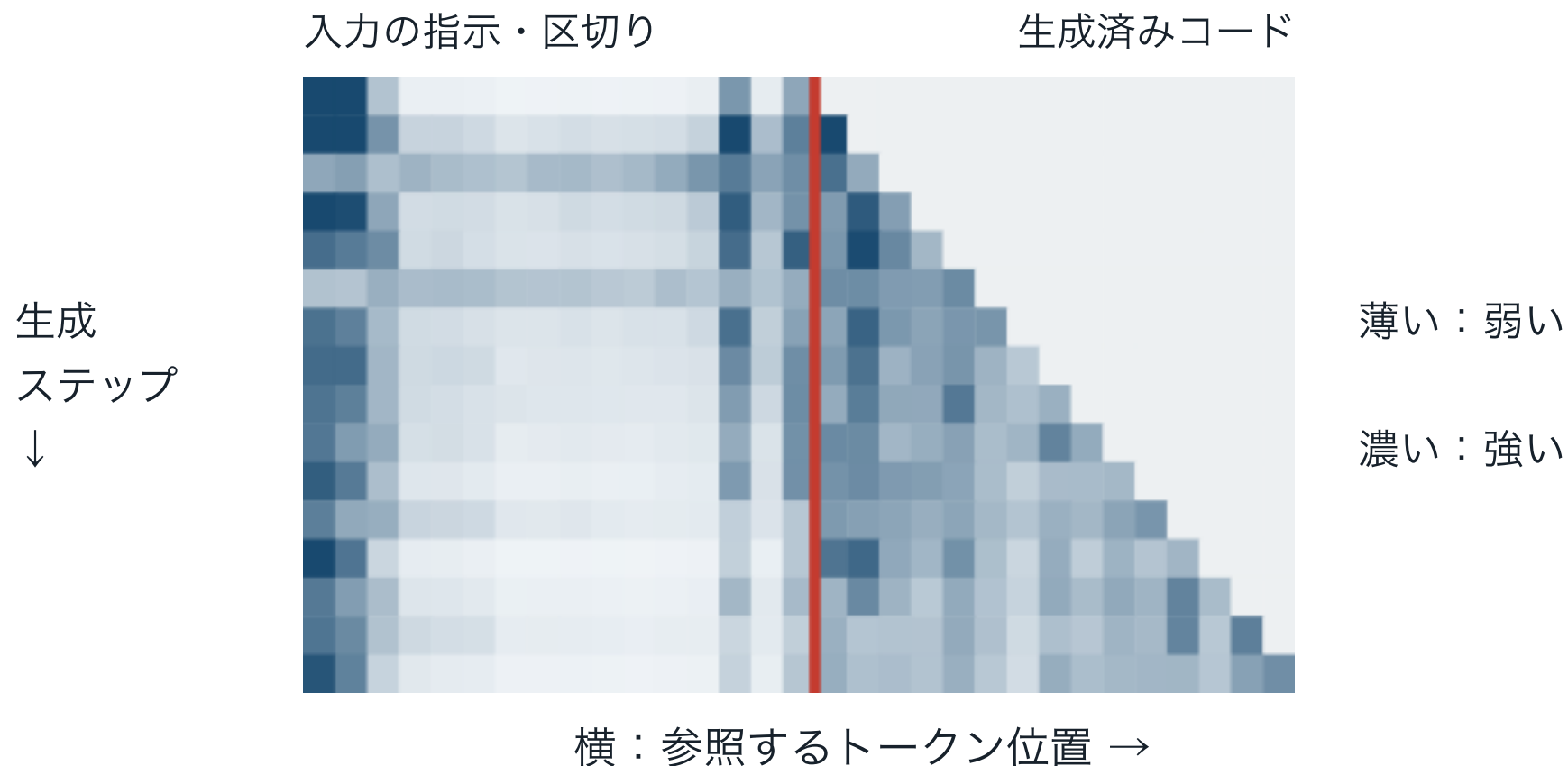
アテンションは、次のコードを作るときに、どこから情報を読むかを決める仕組み。

参照できる情報	内容
日本語の指示	「偶数」「残す」など、求める処理
すでに生成したコード	変数名、構文、処理の続き
開始の印など	入力の始まりや区切り

参照割合だけでは、その情報が必要だったかは確定できない。

情報の受け渡しを止めて、生成結果が変わるかを調べる。

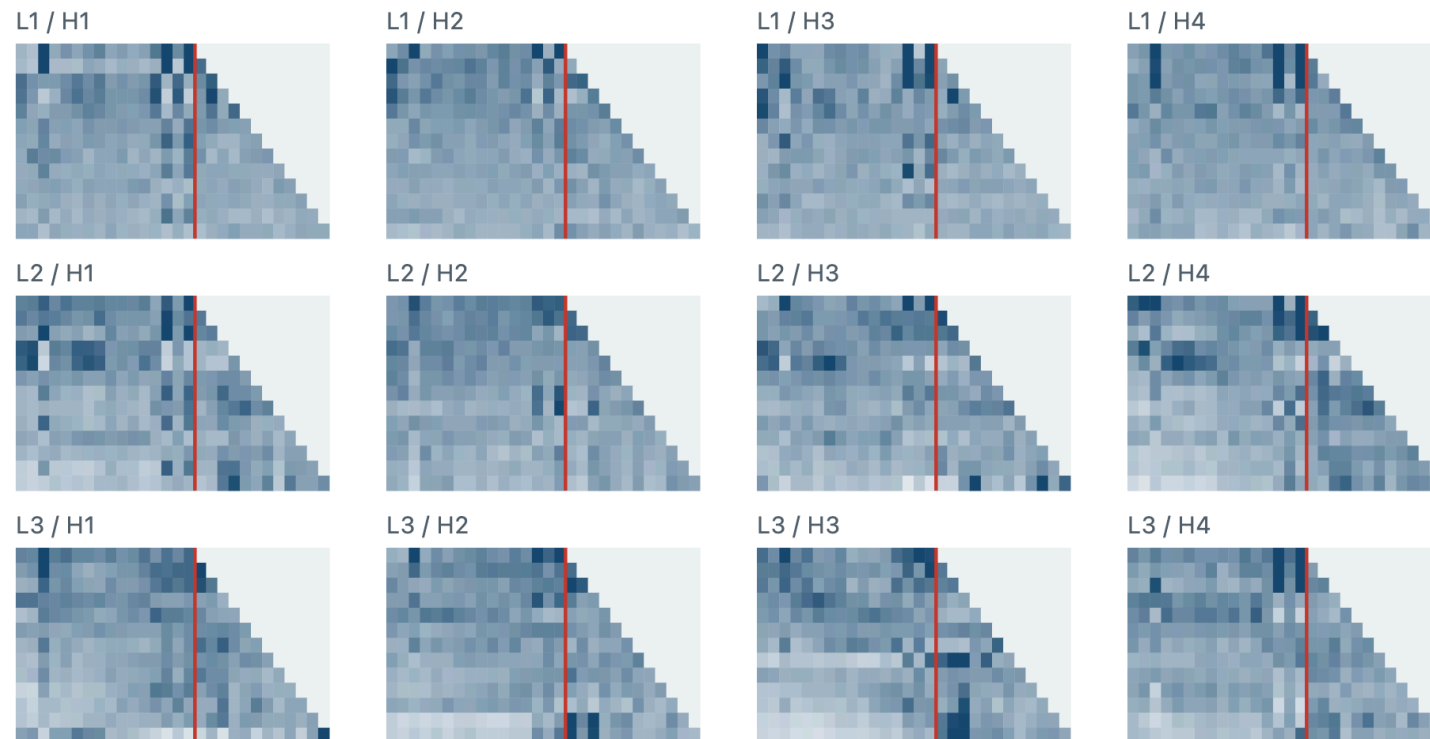
アテンションの図の読み方



濃い位置ほど強く参照。15M・1エポックのL8・H5の例

1M : attention の可視化

同じ指示：各要素を絶対値にして、降順に並べる

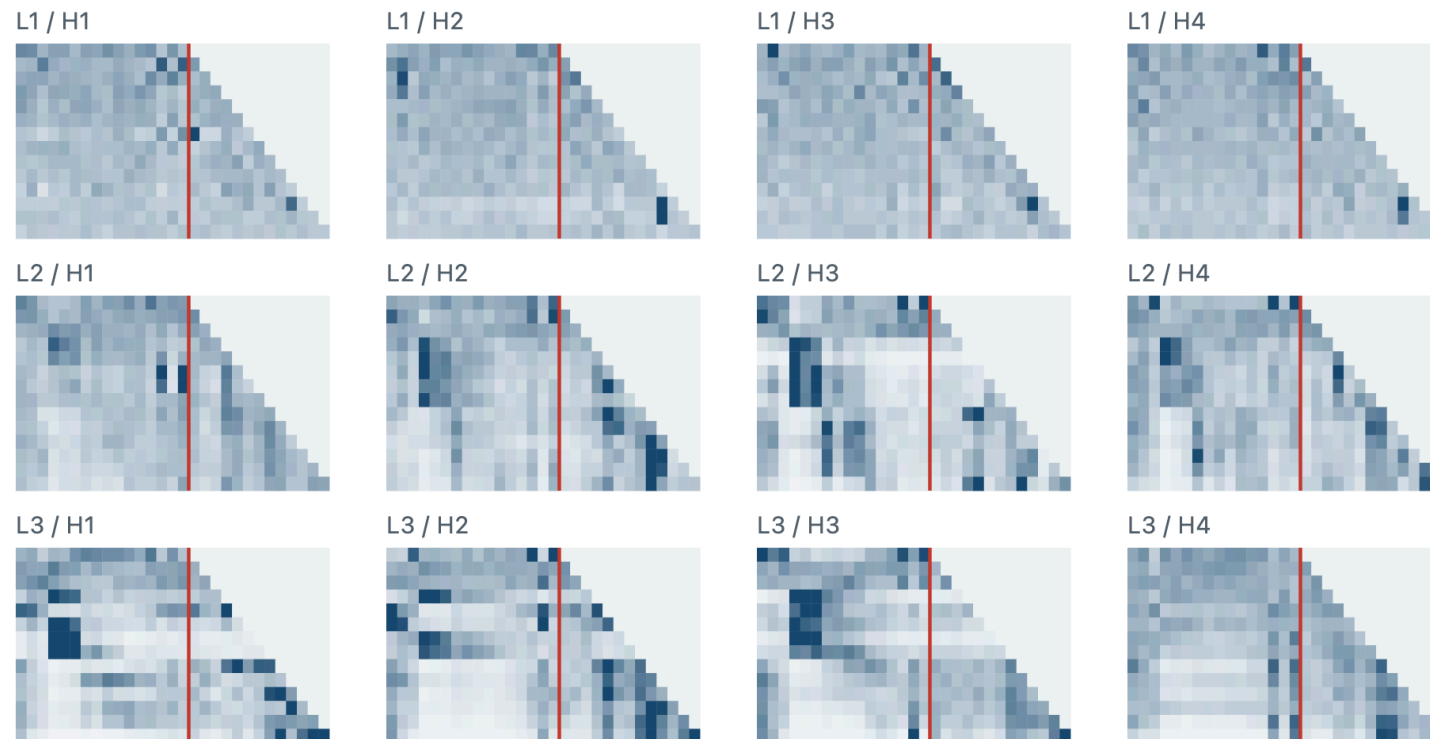


生成結果：未定義の result を参照し、この指示では実行できなかった。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

5M : attention の可視化 (L1~L3)

同じ指示：各要素を絶対値にして、降順に並べる



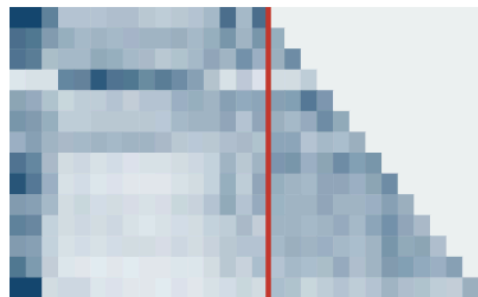
生成結果：絶対値の処理は生成したが、k加算が入り、降順の処理が抜けた。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

5M : attention の可視化 (L4~L5)

同じ指示：各要素を絶対値にして、降順に並べる

L4 / H1



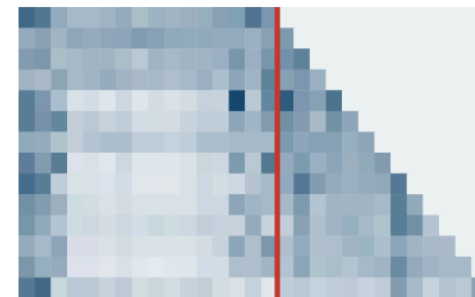
L4 / H2



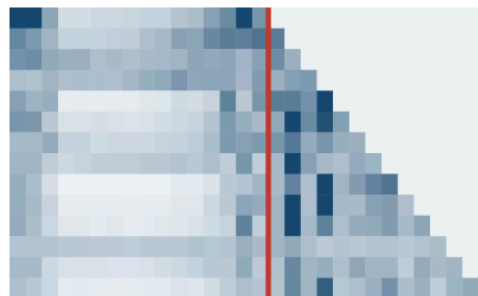
L4 / H3



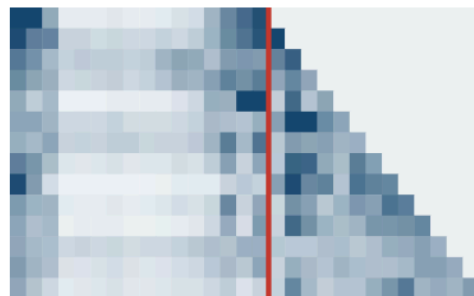
L4 / H4



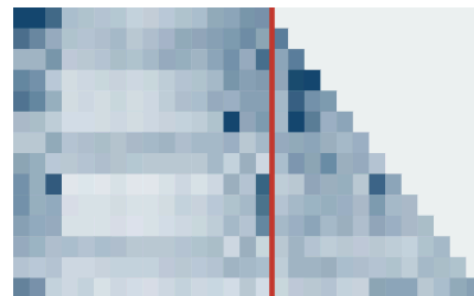
L5 / H1



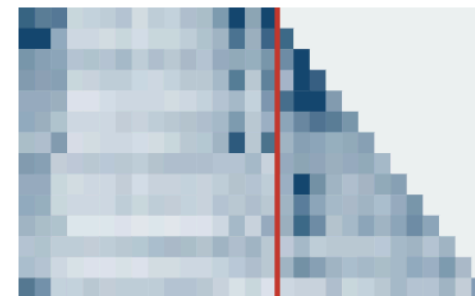
L5 / H2



L5 / H3



L5 / H4

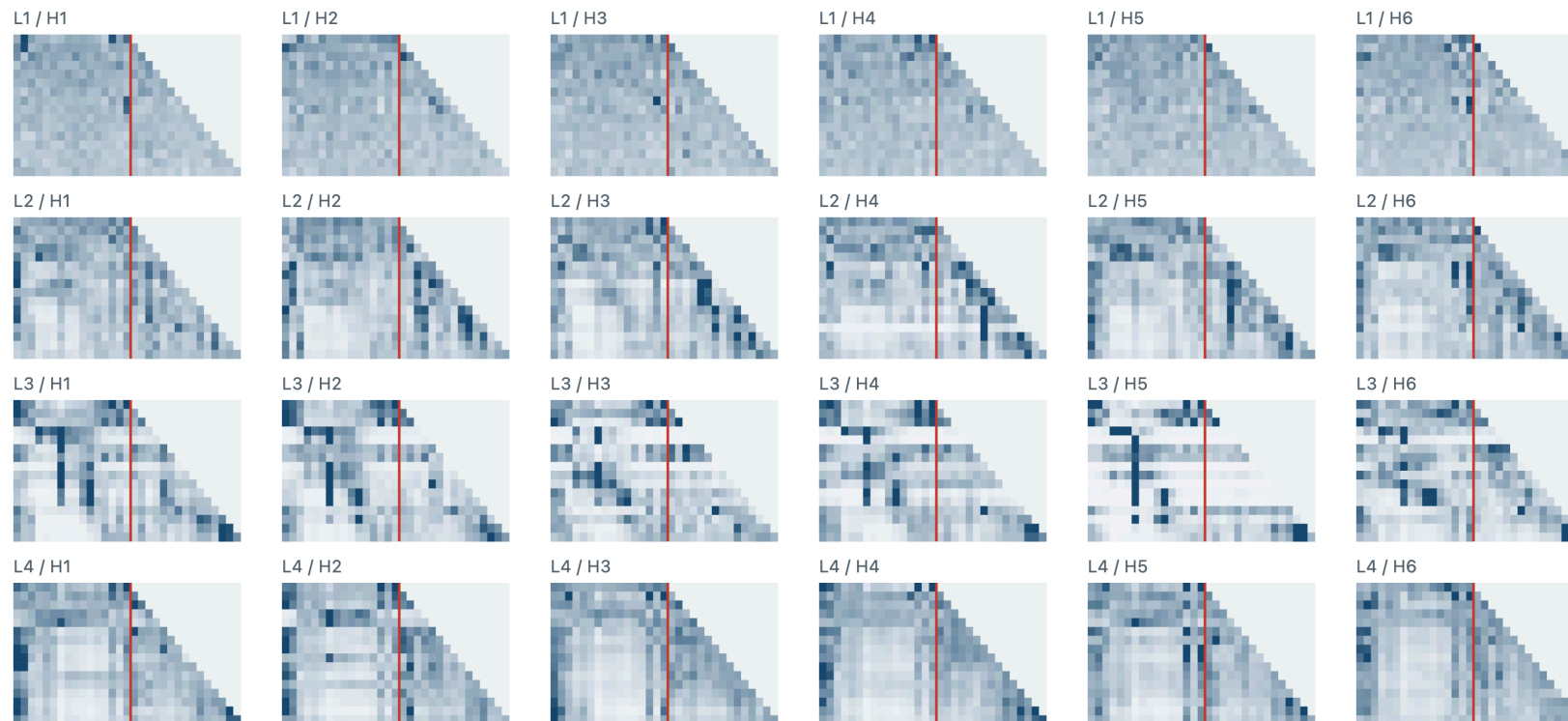


生成結果：絶対値の処理は生成したが、k加算が入り、降順の処理が抜けた。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

15M : attention の可視化 (L1~L4)

同じ指示：各要素を絶対値にして、降順に並べる

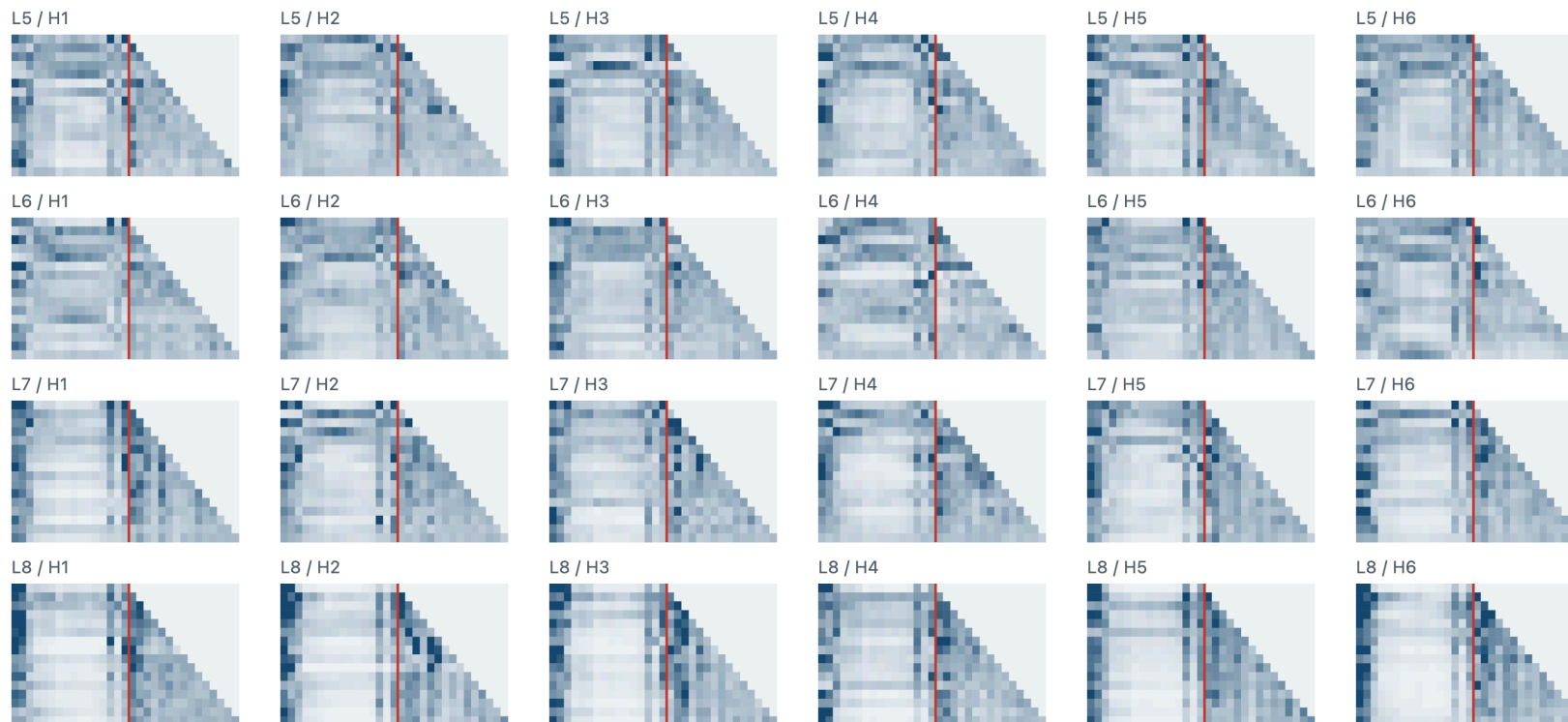


生成結果：絶対値を取って降順に並べるコードを生成した。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

15M : attention の可視化 (L5~L8)

同じ指示：各要素を絶対値にして、降順に並べる



生成結果：絶対値を取って降順に並べるコードを生成した。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

指示の情報を止めると合格数が落ちる

1 epochモデル。学習時に準拠した文型で、24操作を診断。

モデル	通常の生成	指示の情報を止めた生成
1M	3 / 24	0 / 24
5M	22 / 24	0 / 24
15M	22 / 24	2 / 24

指示から受け取る情報が、操作の選択に寄与している。

24操作の内部診断。冒頭変更の14操作、固定5テストとは別。

評価では、表現を変えた場合も確かめる

調べること	今回、分かったこと
決めた文型での動作	5M・15Mは高い全体合格率を示した
未学習の言い換え	15Mでも30件中15件の合格
指示の冒頭の変更	5Mは14件中12件から1件へ低下

一種類のプロンプトでの精度に加え、表現を変えても性能が保たれるかを評価する必要がある。

組み合わせ汎化ベンチマークの前提

各操作を表す日本語表現は、Bokuの訓練データに登場する。
ただし、完成した指示文は訓練データと完全一致しない。

評価872問と訓練192,900件を照合し、指示全文の一致は0件。

課題	訓練と評価で何が変わるか
複数操作で学んだ表現を1操作で使う	複数操作の指示でだけ使った言い方を、単独操作に使う
初めて組み合わせる2操作	訓練では同じ指示に登場しない2操作を組み合わせる
3操作で学んだ順序を2操作で使う	3操作中の既知の順序を、未学習の2操作列として使う

未知の日本語表現への強さとは分けて、**既知の表現を異なる構成で使う能力**を調べる。

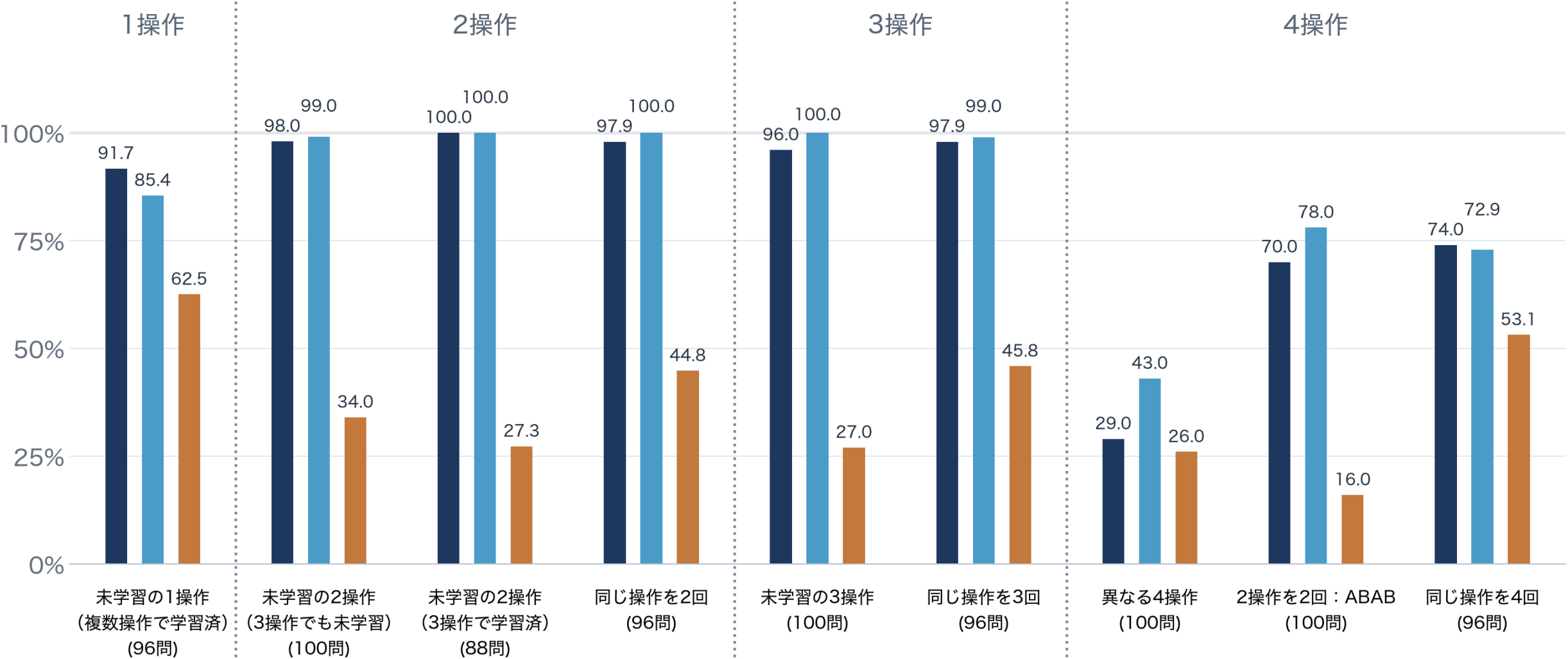
データセットの構成：9種類・872問

操作数	問題の種類	問数	構成
1	複数操作で学んだ表現を1操作で使う	96	24操作 × 4表現
2	初めて組み合わせる2操作	100	5ペア × 両順序 × 10表現
2	同じ操作を2回	96	24操作 × 4表現
2	3操作で学んだ順序を2操作で使う	88	88種類の操作列
3	初めてのペアを含む3操作	100	100種類の操作列
3	同じ操作を3回	96	24操作 × 4表現
4	同じ操作を4回	96	24操作 × 4表現
4	異なる4操作を組み合わせる	100	100種類の操作列
4	2操作の組を2回繰り返す	100	A・B・A・Bの100種類

各問題でコードを1つ生成。**179通りの共通テスト入力ですべて合格した問題の割合**を正答率とする。

問題カテゴリ別のモデル比較

■ Boku 15M・1ep ■ Boku 1M・3ep ■ Qwen3-1.7B



高得点の課題と、残る課題を分けて読む

問題の種類	Boku 15M・1ep	Boku 1M・3ep	Qwen3-1.7B
初めて組み合わせる2操作	98 / 100	99 / 100	34 / 100
初めてのペアを含む3操作	96 / 100	100 / 100	27 / 100
異なる4操作を組み合わせる	29 / 100	43 / 100	26 / 100

- 今回の2・3操作の課題では、小規模モデルにも有望な結果がある。
- 4操作では誤りが多く残る。どの操作を落とすか、順序を変えるかを分析する。
- 1Mは3epoch、15Mは1epoch。生成条件も異なり、規模だけの優劣とは解釈しない。

今回の結果の意義と、次の研究課題

今回のベンチマークでは、小規模モデルの組み合わせ汎化について有望な結果が得られた。

今後は比較条件を揃え、より大きなモデルとの比較を追加するとともに、**評価設計と作成手順を明文化する。**

- 比較条件：訓練回数、プロンプト、生成上限などを記録し、揃えられる条件を揃える。
- 評価設計：何を既知・未知とするか、どの誤りを検出したいかを定める。
- 作成手順：問題の選定、訓練との照合、採点方法、再現手順を公開可能な形に整理する。

ベンチマークの役割

モデルの能力と限界を観測し、次に取り組む研究課題を明確にする。

- 入出力の成功例だけでは、能力を獲得したか判断できない。共通の基準で定量評価する。
- 正誤を検証できる課題では、評価基準を明確にしやすい。小規模モデルでも改善を確かめられる。
- 得点だけでなく、どの条件で成功し、どの条件で失敗するかを見る。

今回なら、**2・3操作での高得点と、4操作で残る誤り**が、次の分析対象を教えてくれる。

自分の研究でベンチマークを作るには

1. 測りたい能力を先に決める。 例：既知の操作を、新しい組み合わせで実行できるか。
2. 目的に沿って課題と正解基準を作る。 何を変え、何を固定するかを決める。
3. 共通条件で測り、成功と失敗を分析する。 作成手順と比較条件も記録する。
4. 高得点で差が見えなくなったら、評価の役割を見直す。 元の結果を残し、次に測る能力を定める。

得点を上げる研究と、能力向上を観測できる評価を作る研究を、両方考える。

この結果が示す範囲

- 訓練は24操作を最大3つ組み合わせるコード生成。追加評価では4操作まで調べた。
- 冒頭変更の比較は14操作。日本語能力全体の測定ではない。
- 学習は一つの乱数設定。繰り返し実験のばらつきは未確認。

今後は文型の偏りを減らした教材で、表現への強さを再評価する。

まとめとWebデモ

限定したコード生成は、小さなモデルでも1 epochで学習できた。

表現を変えた場合の性能と、操作を組み合わせる能力は、分けて確かめる必要がある。

デモの流れ	役割
① 自由な日本語を入力	作りたい処理を伝える
② 補助モデルで文型を整える	対象の操作と表現にそろえる
③ 指示を確認して生成する	Boku-nanoがPythonコードを作る